

TGNS: A transformer-based graph neural network for stock trend forecasting

Haichao Du^{a,b,c,*}, Li Lv^{a,c}, Hongliang Wang^{a,c}, An Guo^d

^a Shenyang Institute of Computing Technology, Chinese Academy of Science, Shenyang, 110168, Liaoning, China

^b University of Chinese Academy of Sciences, 100049, Beijing, China

^c Liaoning Province Digital Twin-based Interactive System Engineering Research Center, Shenyang, 110168, Liaoning, China

^d School of Computer & Information Engineering, Anyang Normal University, Anyang, 455000, Henan, China

ARTICLE INFO

Keywords:

Graph neural network
Transformer
Attention
Stock trend forecasting

ABSTRACT

Advancements in computational technology and financial theory have accelerated quantitative investment research, with stock trend forecasting emerging as a critical challenge. While deep learning models have shown promise, many rely exclusively on market data, ignoring fundamental factors (e.g., financial health, industry trends) and dynamic concept-stock correlations. This paper presents TGNS, a Transformer-based Graph Neural Network framework that addresses these limitations. TGNS integrates a Concept Graph Attention (CGA) module and Stock Graph Attention (SGA) module to model time-varying concept dependencies and global stock interactions, mitigating over-smoothing in traditional graph networks. A novel Weighted Robust Loss (WRL) prioritizes high-return stocks by exponentially weighting cross-sectional returns, enhancing long-short strategy performance. Experiments on CSI100/300 and NASDAQ-100 datasets demonstrate TGNS outperforms state-of-the-art methods (e.g., PatchTST, HIST) by 12.3–21.9% in Information Coefficient (IC). Ablation studies validate the dual-attention architecture's effectiveness in capturing complex market dynamics, confirming TGNS's superiority in dynamic financial environments.

1. Introduction

The stock market is a complex and dynamic environment, and investors seek to make informed decisions by predicting stock trends. Accurate stock trend prediction is crucial for investment strategies and risk management [1]. Traditional stock trend prediction methods often rely on various techniques such as technical analysis, fundamental analysis, and time series analysis. However, these methods often struggle to capture the intricate relationships and dynamic characteristics of stock market data. Many existing studies [2,3] assume that prices of different stocks are independent of each other and build prediction models based solely on time series of information related to each stock, such as historical prices and trading volume (e.g., opening price, closing price, highest price, and volume).

In practice, the price trends of different stocks are often highly correlated with each other, and these stocks are associated with common concepts. For example, over the last year, with the popularity of ChatGPT, stocks related to the “GPT concept” in the A-share market have continued to rise. Companies listed this year that are associated with the ChatGPT concept have seen significant increases

* Corresponding author at: University of Chinese Academy of Sciences, 100049, Beijing, China.

E-mail addresses: duhaichao20@mails.uicas.ac.cn (H. Du), lli@sict.ac.cn (L. Lv), wanghl@sict.ac.cn (H. Wang), guoan@aynu.edu.cn (A. Guo).

<https://doi.org/10.1016/j.ins.2025.122555>

Received 28 March 2025; Received in revised form 25 July 2025; Accepted 25 July 2025

in their stock prices. Another example is during the outbreak of the COVID-19 pandemic, medical stocks related to COVID-19 testing experienced a surge, and they subsequently declined as the epidemic dissipated, indicating a high correlation among different stock price trends. These concepts are often derived from publicly available company information on the internet and can be specified through various dimensions such as industry, sector, and business. Recently, studies have started utilizing valuable information from these stock concepts for stock trend prediction. For instance, some direct methods [4,5] use known prior stock concepts as input features for linear prediction models. Additionally, other methods [6–9] that utilize Graph Neural Networks (GNNs) [10] to capture the interconnections and dependencies among stocks, enabling a more comprehensive analysis of the market. GNNs can represent stocks as nodes in a graph and model the relationships between them as edges, allowing these models to leverage rich information embedded in the network structure. The core idea of GNNs is to learn node representations through interactions among nodes. They capture the relationships and global structure among nodes by performing information propagation and aggregation in the graph. GNNs typically employ an iterative approach where each iteration updates the representations of the nodes, enabling them to gradually focus on the information from their neighboring nodes. This concept is often referred to as neighborhood aggregation.

However, these methods often directly apply GNNs to the financial market, limiting the expressive power of GNNs and failing to fully harness the value of stock concepts. Through our research analysis and experiments, we have found that over-smoothing [11,12] and over-compression [13] constrain the performance of GNNs in the application to the stock market. Over-smoothing occurs when, after several layers, all node representations converge to a constant value, while over-compression happens when information from distant nodes fails to effectively propagate to certain “bottleneck” areas in the graph, as excessive information gets compressed into a fixed-length vector. Therefore, designing new architectures that go beyond neighborhood aggregation is crucial for more effective application of GNNs in financial markets.

In addition, most previous studies assumed that these connections of known concepts in GNNs are static, and therefore information propagates through these connections according to the same pattern. However, in real financial markets, a stock may have dynamic correlations with various concepts. For example, Jiu'an Medical* has two prior concepts: “medical devices,” which it shares with Yingke Medical*, and “antigen testing”, which it shares with Shuoshi Biology* (* these are all stocks listed on the A-share market). Clearly, during the lockdown period caused by the COVID-19 pandemic, the upward trend in Jiu'an Medical's stock price was primarily driven by its association with the surging “antigen testing” concept rather than the “medical devices” concept. Therefore, how to construct a dynamic concept representation network is also key for GNN to predict stock price trends.

In this paper, we propose a new framework called Transformer Graph Network for Stock Trend Forecasting (TGNS) to address the GNN limitations described above and enhance the model's representational capabilities in terms of leveraging stock concepts. Unlike prior GNN-Transformer hybrids, TGNS uniquely integrates Concept Graph Attention (CGA) and Stock Graph Attention (SGA) to model dynamic concept-stock correlations. This resolves over-smoothing via residual structures while prioritizing high-return stocks through our novel Weighted Robust Loss (WRL). TGNS leverages the advantages of Transformers in capturing global dependencies and modeling long-range interactions, complementing the local neighborhood information captured by GNNs. By introducing Transformers, GNNs can integrate global information by considering attention over the entire graph, rather than being limited to local neighborhoods. This allows GNNs to leverage global structural patterns and dependencies, improving representation learning. TGNS introduces a more flexible way of modeling relationships between nodes through self-attention mechanisms, enabling GNN to dynamically capture complex dependencies and interactions among nodes, surpassing the limitations of traditional message-passing schemes.

Specifically, TGNS contains a concept graph attention (CGA) module and a stock graph attention (SGA) module that captures pairwise interactions between all stocks and establishes connections between concept information and stocks. Thus, TGNS can effectively learn both local and global trading information while alleviating the over-smoothing phenomenon. CGA module focuses on capturing relationships and interactions among different concepts or factors related to stocks. It uses attention mechanisms to assign weights to the connections between concepts, enabling the model to selectively emphasize or de-emphasize certain concepts based on their relevance to the task at hand. By incorporating the CGA module, TGNS can effectively leverage the interplay between various concepts and their impact on stock behavior. Another important module, SGA, can dynamically capture the complex dependencies and interactions among stocks, which computes attention weights between any two stocks, enabling the learning of mutual influences between them. Additionally, it can uncover hidden information that cannot be explained by factor correlations in the original data.

To better optimize our TGNS framework, we propose a new loss function called weighted robust loss (WRL). The main characteristic of the weighted robust loss function is its robustness to outliers. When the difference between predicted and true values is small, WRL function behaves similarly to the mean squared error (MSE) loss function and is not sensitive to outliers. However, when the difference is large, the growth rate of the loss function slows down, resulting in a smaller impact from outliers. By increasing the weight values in the loss function, we can enhance the weights of stocks with higher cross-sectional returns and reduce the weights of stocks with lower cross-sectional returns, thereby improving the long side performance of the model.

To evaluate the effectiveness of our framework, we conducted extensive experiments on real financial datasets. We compared the predictive performance of the proposed TGNS with various different methods, including graph-based and attention-based baselines. The experimental results demonstrate that TGNS performs remarkably well in stock movement prediction. Compared to traditional GNN methods, TGNS can capture complex dependencies and long-range interactions among stocks more accurately, thus improving prediction accuracy. Additionally, ablation studies validate the effectiveness of the individual TGNS modules.

The main contributions of this paper are as follows:

- The paper introduces the Transformer Graph Network for Stock Trend Forecasting (TGNS) framework, which addresses the limitations of traditional GNNs in financial markets. TGNS combines the strengths of GNNs and Transformers to capture both

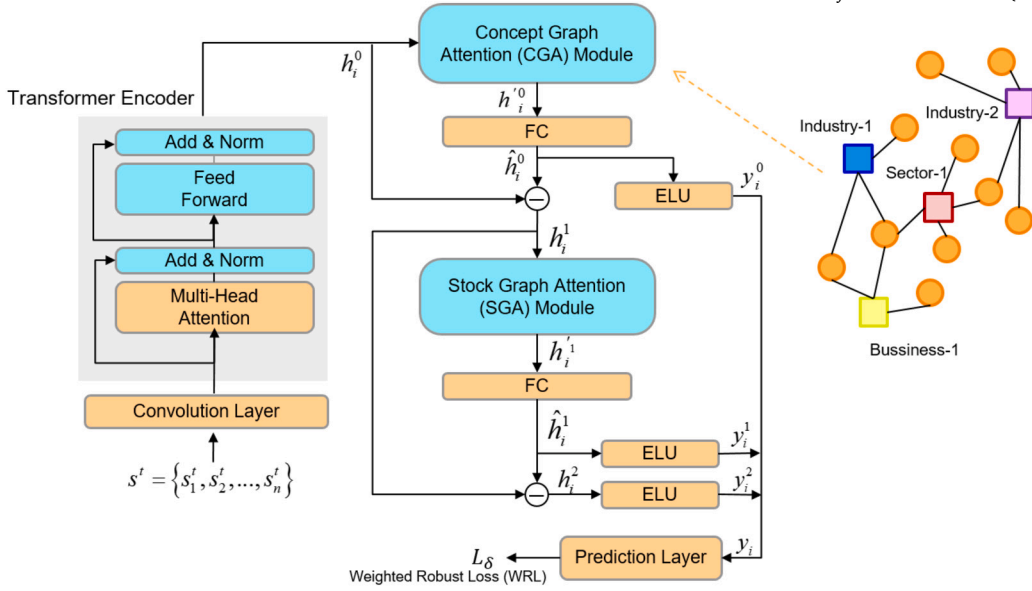


Fig. 1. The framework of TGNS.

local and global dependencies in stock data. By incorporating self-attention mechanisms, TGNS dynamically models complex dependencies and interactions among stocks, surpassing the limitations of traditional message-passing schemes.

- The framework consists of the concept graph attention (CGA) module and the stock graph attention (SGA) module. The CGA module captures relationships and interactions among different stock concepts, allowing the model to selectively emphasize relevant concepts based on their relevance to the prediction task. The SGA module captures complex dependencies and interactions among stocks, uncovering hidden information that cannot be explained by factor correlations in the original data.
- To optimize the TGNS framework, the paper proposes a new loss function called weighted robust loss. By increasing the weight values in the loss, the model can enhance the weights of stocks with higher cross-sectional returns and reduce the weights of stocks with lower cross-sectional returns, improving the model's performance on the long side.
- Extensive experiments on real financial datasets validate the effectiveness of the TGNS framework. It outperforms traditional GNN methods and achieves accurate stock movement predictions by capturing complex dependencies and long-range interactions among stocks.

The rest of this paper is organized as follows: Section 2 reviews the work related to stock prediction. Section 3 introduces the construction and details of the proposed model. Section 4 evaluates the effectiveness of the proposed model and shows the experimental results. Finally, the paper is summarized in Section 6.

2. Related work

In this section, we briefly review recent works on GNNs and Transformers for stock move prediction.

2.1. Graph neural networks (GNNs)

For learning representations of graph-structured data, graph neural networks (GNNs) [14,15] have attracted considerable attention. GNNs operate on data structured as a graph, which consists of nodes (vertices) and edges (connections between nodes). Nodes can represent entities, such as users or products, while edges represent relationships or interactions between entities. They allow for efficient modeling of relationships and interactions between elements within a graph, making them well-suited for various tasks including node classification, link prediction, and graph classification. The first step in GNNs is to assign an initial embedding or feature representation to each node in the graph. These embeddings capture the characteristics or attributes of the nodes and serve as the input to the GNN model. The key concept in GNNs is message passing, where nodes exchange information with their neighbors iteratively [16]. This allows nodes to gather and propagate information throughout the graph, capturing both local and global dependencies. The GNN layer is employed to integrate the representations and graph structure, enhancing the Transformer layer's utilization of the graph structure. Additionally, the GNN layer efficiently captures the information of relevant nodes in the neighborhood through the message passing process [17]. Taking node v_i as an example, the message passing process of the GNN layer can be described as follows:

$$h_u(v_i) = PassMessage(h_k, \forall v_k \in N(v_i)) \quad (1)$$

$$h_i = \text{Aggregate}(h_i, h_u(v_i))$$

where $N(v_i)$ is the neighbor nodes set of v_i , h_i , $h_u(v_i)$ are the representations of v_i , v_k respectively. $\text{PassMessage}(\cdot)$ and $\text{Aggregate}(\cdot)$ are the message passing function and aggregation function defined by GNN layer.

The GNN layer is employed to integrate the representations and graph structure, enhancing the Transformer layer's utilization of the graph structure. Following the message passing mechanism, the GNN layer captures neighborhood information through iterative aggregation and update operations. For node v_i , the k -th layer propagation is defined as:

$$m_{u \rightarrow i}^{(k)} = \phi \left(h_u^{(k-1)}, h_i^{(k-1)}, e_{ui} \right) \quad (\text{Message function}) \quad (2)$$

$$h_i^{(k)} = \psi \left(h_i^{(k-1)}, \bigoplus_{u \in \mathcal{N}(v_i)} m_{u \rightarrow i}^{(k)} \right) \quad (\text{Aggregation \& Update}) \quad (3)$$

- $\mathcal{N}(v_i)$ denotes the neighbor set of v_i
- $h_i^{(k)}$ is the embedding of v_i at layer k
- $\phi(\cdot)$: Message function (e.g., linear projection or attention)
- $\bigoplus$: Permutation-invariant operator (e.g., sum, max, or mean)
- $\psi(\cdot)$: Update function (e.g., GRU or MLP)

This formulation generalizes the intuitive operations described in earlier works by explicitly decoupling message construction (ϕ), neighborhood aggregation ($\bigoplus$), and node state updating (ψ). As shown in Eq. (2)-(3), each node iteratively refines its representation by synthesizing its previous state $h_i^{(k-1)}$ with aggregated messages from neighbors. Compared to the preliminary implementation, our framework additionally incorporates edge features e_{ui} and theoretically guarantees permutation invariance through $\bigoplus$. After the message passing step, each node updates its own embedding by considering its current embedding and the aggregated information from its neighbors. This update can be performed using a neural network layer or a combination of linear transformations and activation functions.

GNNs have demonstrated remarkable performance in a wide range of application [18]. They have been successfully applied to social network analysis, recommendation systems, molecular chemistry, computer vision, and natural language processing, among others. In the last few years GNNs have begun to be used in the field of financial market forecasting. HIST [19] is a novel framework for stock trend forecasting that effectively extracts and utilizes both predefined concepts and hidden concepts. It focuses on mining concept-oriented shared information to enhance the accuracy of stock trend predictions. By leveraging the shared information among stocks and incorporating individual stock information, HIST achieves improved stock trend forecasting performance. MGRA [20], a multi-relational graph attention ranking (MGAR) network. The MGAR model utilizes adaptive learning mechanisms to aggregate information from the multiple graphs, resulting in effective relation embeddings. These embeddings, combined with captured price trend embeddings, enable the MGAR model to generate a ranking list of future returns.

Although GNNs have achieved notable success, they encounter challenges like over-smoothing and limited expressiveness. Over-smoothing occurs when repeated neighborhood aggregation leads to the loss of discriminative information. Limited expressiveness refers to the difficulty in capturing higher-order dependencies and long-range relationships.

2.2. Transformer

The Transformer architecture, originally introduced in machine translation, is rooted in the idea of attention mechanisms [21,22]. Attention allows the model to focus on different parts of the input sequence while processing each position. The attention mechanism in Transformers is often referred to as *self-attention* or *scaled dot-product attention*. The attention mechanism involves three concepts: query, key, and value. These are linearly transformed representations of the input sequence that capture different aspects of the information. The query represents the position in the sequence that we are currently focusing on, while the keys and values represent all other positions. To determine the importance or relevance of each position to the current query, the dot product similarity between the query and the keys is computed. This results in a set of similarity scores, where each score represents the similarity between the query and a particular position in the sequence. The scores quantify the attention or importance that each position receives from the current query. The similarity scores are then scaled using a softmax function to obtain attention weights. The attention weights determine how much focus or weight each position should have in the context of the current query. The weighted sum aggregates the values according to their corresponding attention weights. This produces a context vector or attention output that captures the relevant information from the entire sequence, weighted by their importance. In the Transformer encoder, each layer computes the attention distribution for every token and aggregates information from other tokens based on their attention scores. This mechanism enables each token to gather insights from all other tokens, facilitating the modeling of long-term dependencies in sequential data. The Transformer's exceptional effectiveness in representation learning quickly surpassed Recurrent Neural Networks and convolutional neural networks, establishing itself as the new state-of-the-art model across various tasks [23–26]. Recently, emerging research streams [26–28] have focused on adapting Transformer architectures to graph-structured data, commonly termed graph Transformers. A significant research direction within these approaches involves developing specialized mechanisms that enable transformers to effectively capture graph structural information.

Dwivedi et al. [21] and Kreuzer et al. [29] make use of the eigenvectors derived from the graph Laplacian matrix as a component of input features, aiming to retain the local topological characteristics of nodes. Bo et al. [30] employ the Transformer architecture to maintain both the magnitudes and the relative discrepancies among all eigenvalues of the graph Laplacian. Meanwhile, Jain et al. [31] and Chen et al. [26] have developed hybrid neural network frameworks. These frameworks first apply GNN layers to extract the structural information contained in graphs and then use Transformer layers to facilitate the learning of node representations.

Recent breakthroughs in stock trend forecasting have shifted toward Transformer-based architectures and federated learning to address dynamic market dependencies and non-IID data challenges. Nie et al. [32] introduces a patch-based Transformer that partitions stock price time series into local segments to capture multi-scale patterns. While effective for high-frequency data, its static patch structure struggles to adapt to abrupt concept shifts, such as the rapid emergence of AI-driven stock clusters in 2023. Zhou et al. [33] enhances long-range dependency modeling via adaptive sparse attention but remains limited to sequential analysis, neglecting cross-sectional concept-stock correlations. These methods highlight the need for frameworks that dynamically integrate concept-level semantics with fine-grained stock interactions—a gap addressed by TGNS through its dual CGA-SGA architecture (Section 3).

3. Methodology

3.1. Overview

To begin, we present an overview of the TGNS framework. The inputs of date t comprises of the stock features $s^t = \{s_1^t, s_2^t, \dots, s_n^t\}$ of n stocks. These features encompass a range of information related to stocks. Our objective is to predict the future trend of individual stocks. It is important to note that each stock feature, denoted as s_i^t , represents a sequence of historical raw features such as opening prices, closing prices, trading volume, and so on. The framework of TGNS is shown in Fig. 1, which consists of four important components: (1) Transformer Encoder module, (2) Concept Graph Attention (CGA) module, (3) Stock Graph Attention (SGA) Module, and (4) Weighted Robust Loss.

Transformer Encoder: This module is responsible for expanding the model's receptive field and efficiently aggregating attention sampling information. It starts by passing the input data through a convolutional layer, which transforms the feature vectors to smooth out temporal features. Then, it utilizes attention-based encoders to process the input data, enabling the model to capture the stock feature of time series $\{s_1^t, s_2^t, \dots, s_n^t\}$.

Concept Graph Attention (CGA) module: The data processed by Transformer encoder is passed to the Concept Graph Attention module. In this module, the encoded features are then fed into a masked self-attention layer, allowing the model to learn the returns that explain the interconnections and correlations between various concepts in the stock market, such as industry or sector relationships. The output obtained after the calculation of the masked self-attention layer is then subjected to residual processing with the output of the Graph Transformer module. CGA aims to extract the information that cannot be explained by sector or industry correlations, representing the residual information from the original data. CGA's sector/industry-based graph construction (Equation (6)) and residual learning (Equation (9)) disentangle concept-driven returns without explicit knowledge graphs. The ELU activation (Equation (7)) enriches nonlinear interactions, outperforming ReLU (Table 4).

Stock Graph Attention (SGA) module: After obtaining the residual hidden state output from CGA module, it is used as input to the Stock Graph Attention module, where the attention scores between any two stocks are computed. This module employs a global self-attention layer to learn the returns that elucidate the factor correlations between stocks. It leverages a simple yet effective algorithm to extract hidden concepts and their corresponding representations. The residual representation from the previous module is combined with the factor itself to explain specific returns, incorporating both local and global information.

Weighted Robust Loss: The Weighted Robust Loss is a loss function utilized for model optimization. Its purpose is to enhance the model's resilience to outliers present in the training data. This loss function assigns weights to different samples, dynamically adjusting the significance of each sample during training. By doing so, it mitigates the impact of outliers on the model's performance and enhances its stability.

Overall, the TGNS model combines the power of Transformer-based encoders, attention mechanisms, and graph neural networks to capture both local and global dependencies in stock data. The Weighted Robust Loss helps the model handle outliers effectively during training, leading to improved prediction performance.

3.2. Transformer encoder module

The Transformer method is a popular architecture for sequence-to-sequence tasks in natural language processing (NLP) [34]. Transformer relies on self-attention mechanisms to capture the relationships and dependencies between different positions in the input sequence. The self-attention mechanism computes the attention weights for each position in the input sequence. Given a sequence of hidden states $\mathbf{H} = [\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_n]$, the self-attention operation can be defined as follows:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \quad (4)$$

where $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are linearly projected queries, keys, and values, respectively. d_k represents the dimension of the keys.

We utilize Transformer Encoder to extract the time-series features. First, we smoothed the input stock time-series data through a layer of convolutional neural network layer. Then, the encoder takes the input sequence and applies a stack of identical layers,

each having two sub-layers. The first sub-layer is a multi-head self-attention mechanism that allows the encoder to attend to different positions in the input sequence. The second sub-layer is a position-wise fully connected feed-forward network that processes the outputs of the self-attention layer. The encoder employs residual connections and layer normalization to facilitate training and improve the flow of information through the network. It is trained using a variant of the attention mechanism called “scaled dot-product attention” and a modified form of the “transformer architecture objective” known as the “label smoothing” technique. To capture different types of information, the encoder employs multiple parallel attention layers. The outputs of these layers are concatenated and linearly transformed to obtain the final attention output. Mathematically, the multi-head attention operation can be expressed as:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) \mathbf{W}^O \quad (5)$$

where $\text{head}_i = \text{Attention}(\mathbf{QW}_i^Q, \mathbf{KW}_i^K, \mathbf{VW}_i^V)$ represents the i -th attention head, and $\mathbf{W}_i^Q$, $\mathbf{W}_i^K$, $\mathbf{W}_i^V$, and $\mathbf{W}^O$ are learnable weight matrices.

Besides, the encoder in TGNS applies a position-wise feed-forward network to each position in the sequence independently. The feed-forward network consists of two linear transformations with a ReLU activation function in between. The formula for the position-wise feed-forward network can be written as:

$$\text{FFN}(\mathbf{X}) = \text{ReLU}(\mathbf{XW}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2 \quad (6)$$

where $\mathbf{X}$ represents the input sequence, $\mathbf{W}_1$, $\mathbf{W}_2$, $\mathbf{b}_1$, and $\mathbf{b}_2$ are weight matrices and bias vectors.

Let s_i^t stand for the feature of stock i on date t , which consists of l -dimensional historical data including stock prices and trading volumes. Given the set of stock features $s^t = \{s_1^t, s_2^t, \dots, s_n^t\}$ for n stocks at time t , the purpose of the stock feature encoder is to convert these features into low-dimensional representations that capture the essential information of each stock. Due to the transformer's exceptional capability in modeling long-range dependencies, this paper adopts a transformer encoding network with a hidden dimension d as the stock feature encoder. Specifically, the final hidden state from the transformer's output is taken as the initial embedding of stocks at date t :

$$x_i^{t,0} = \text{transformerEncoding}(s_i^t), \quad (7)$$

where $x_i^{t,0}$ represents the initial embedding of stock i . The initial embeddings of all stocks at date t form a matrix denoted as h_i^0 , where the i -th row corresponds to $x_i^{t,0}$.

3.3. Concept graph attention (CGA) module

The concepts refer to human-defined categories associated with a stock, including the stock's sector, industry. TGNS includes a concept graph attention (CGA) module, which is built upon a graph neural network that extracts the shared information of stocks based on the prior concepts. CGA requires constructing a graph for the stocks, which involves creating an $N \times N$ adjacency matrix W for the N stocks in the stock pool. We adopt a relatively simple approach based on sectors or primary industries. Taking industry-based graph construction as an example, if two stocks i and j belong to the same industry, then $W_{ij} = W_{ji} = 1$; otherwise, it is 0. The assumption underlying industry-based graph construction is that there is only mutual influence among stocks within the same industry. However, in reality, there can be inter-dependencies between stocks belonging to different industries, especially those positioned upstream or downstream in the industrial chain.

CGA performs a linear transformation on h_i to obtain $W^0 h_i^0$; apply self-attention mechanism π_0 to obtain attention scores e_{ij}^0 representing the influence of stock j on stock i . The attention mechanism concatenates the features of stocks i and j , followed by linear transformation and ELU activation.

$$e_{ij}^0 = \pi_0(W^0 h_i^0, W^0 h_j^0) = \text{ELU}(a^0 [W^0 h_i^0 || W^0 h_j^0]) \quad (8)$$

$$\text{ELU}(x) = \begin{cases} x & \text{if } x \geq 0 \\ \alpha \cdot (e^x - 1) & \text{if } x < 0 \end{cases} \quad (9)$$

In the above ELU equation, a controls the behavior of the function for negative inputs. It determines the slope of the function as it approaches negative infinity. The parameter a is typically set to a small positive value, such as 1, to ensure a smooth transition and avoid abrupt changes in the function. By introducing the a parameter, the ELU activation function aims to address the vanishing gradient problem that can occur with other activation functions, such as the standard ReLU (Rectified Linear Unit) [35]. The negative saturation range controlled by a allows the ELU function to have non-zero gradients for negative inputs, which can help improve the flow of gradients during backpropagation and potentially enhance the learning process. We build a graph based on sectors or industries to analyze stocks, where two stocks are considered neighbors if they belong to the same sector or industry. For a stock i its neighbor $j \in \mathcal{N}(i)$, attention scores e_{ij}^0 are softmax normalized to obtain attention weights α_{ij}^0 :

$$\alpha_{ij}^0 = \text{softmax}_j(e_{ij}^0) = \frac{\exp(e_{ij}^0)}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik}^0)}. \quad (10)$$

For stock i the hidden states of all neighboring stocks are weighted summed based on attention weights. The resulting sum is then passed through an ELU activation function. Finally, the updated hidden state of stock i denoted as h_i^0 , is obtained by adding its own hidden state h_i^0 to the sum.

$$h_i^0 = h_i^0 + \text{ELU}\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^0 W^0 h_j^0\right) \quad (11)$$

In Equation (11), W corresponds to the weight matrix applied for linear transformation of the neurons in the hidden layer. CGA contains a residual architecture: the hidden state h_i^0 is passed through a fully connected layer to obtain $\hat{h}_i^0$. Firstly, the residual between h_i^0 and $\hat{h}_i^0$ is calculated, resulting in a new hidden state h_i^1 . This new hidden state represents information that cannot be explained by sector or industry association, similar to sector or industry neutralization. Subsequently, an ELU activation is applied to $\hat{h}_i^0$, yielding y_i^0 , which represents the return representation that can be explained by sectors or industries (concepts) association.

The core innovation of our CGA module lies in its ability to model *dynamic concept correlations*, which addresses the limitation of static graph structures in traditional GNNs. As shown in Eq. (8)-(11), the attention weights α_{ij}^0 are recomputed at each time step based on current stock embeddings, allowing the model to adapt to evolving market conditions. For instance, consider two medical stocks: Stock A (medical devices) and Stock B (antigen testing). During normal periods, their correlation might be weak ($\alpha_{ij}^0 \approx 0.1$). However, during a pandemic outbreak (e.g., COVID-19), the model automatically increases attention weight to antigen testing concepts ($\alpha_{ij}^0 \uparrow 0.8$) through the ELU-activated attention mechanism in Eq. (8).

3.4. Stock graph attention (SGA) module

The stock graph attention module mainly consists of global self-attention, residual network, and output layer. The difference between stock graph attention module and the previously mentioned concept graph attention module lies in the following: concept graph attention module calculates attention weights only between stocks belonging to the same sector or industry, whereas stock graph attention module calculates attention weights between any two stocks. The purpose of the global self-attention layer is to learn the mutual influence between any pair of stocks. The stock graph attention module, as described, incorporates global self-attention, a residual network, and an output layer. Its main distinction from the concept graph attention module lies in the calculation of attention weights. In the concept graph attention module, attention weights are computed only between stocks that belong to the same sector or industry. Overall, the stock graph attention module extends the concept graph attention module by considering attention weights between any two stocks and incorporating a global self-attention layer to capture the mutual influence between stocks. The Stock Graph Attention (SGA) module in TGNS dynamically captures complex dependencies among stocks by adaptively recalibrating edge weights based on both historical price patterns and cross-sectional fundamental correlations. Specifically, SGA employs a multi-head self-attention mechanism to compute pairwise affinity scores between stocks, which are then fused with the predefined industry-sector graph to generate context-aware adjacency matrices (Eq. (11)). This hybrid approach enables the model to discern transient market sentiment-driven linkages (e.g., momentum spillovers) alongside structural sector-based connections, addressing the rigidity of static graph topologies in conventional GNNs. Additionally, the residual architecture enables self-attention heads to dynamically recalibrate their focus across market regimes. For instance, during sector-agnostic risk events, the residual pathways allow attention heads to amplify ESG-factor dependencies while suppressing transient sector correlations, a capability absent in static graph constructions. By iteratively blending residual features (localized pairwise dependencies) with self-attention outputs (global context-aware influences), the SGA module constructs hierarchical representations. This fusion reveals latent transmission pathways that manifest through indirect, nonlinear channels undetectable via linear correlation metrics. Similar to the masked self-attention in the concept graph attention module, we first calculate the attention scores e_{ij}^1 between any two stocks i and j based on the hidden state h_i^1 .

$$e_{ij}^1 = \pi_1(W^1 h_i^1, W^1 h_j^1) = \text{ELU}(a^1[W^1 h_i^1 \mid |W^1 h_j^1]) \quad (12)$$

Afterward, softmax normalization is applied to obtain attention weights a_{ij} :

$$a_{ij}^1 = \text{softmax}_j(e_{ij}^1) = \frac{\exp(e_{ij}^1)}{\sum_k \exp(e_{ik}^1)} \quad (13)$$

For stock i the hidden states of all stocks are weighted summed based on attention weights, followed by a ELU activation. Finally, the updated hidden state of stock i denoted as h_i^1 , is obtained by adding its own hidden state h_i^1 to the sum.

$$h_i^1 = h_i^1 + \text{ELU}\left(\sum_j a_{ij}^1 W^1 h_j^1\right) \quad (14)$$

Residual Structure: The hidden state h_i^1 is passed through a fully connected layer to obtain $\hat{h}_i^1$. Firstly, the residual between h_i^1 and $\hat{h}_i^1$ is calculated, resulting in a new hidden state h_i^2 . This new hidden state represents information that cannot be explained by factor association, similar to factor neutralization. Subsequently, an ELU activation is applied to $\hat{h}_i^1$, yielding y_i^1 , which represents the return representation that can be explained by factor association. **Output Layer:** The hidden state h_i^2 is passed through an ELU activation, resulting in y_i^2 , which represents the specific return representation explained by factors. The three parts of return representations,

y_i^0 , y_i^1 , and y_i^2 , are summed together to obtain the aggregated return representation y_i . This aggregated representation is then passed through the output layer to ultimately obtain the return prediction for stock i .

3.5. Weighted robust loss

Financial market data is full of noise, and outliers may seriously affect the generalizability of the model, which in turn may cause a decrease in the return of out-of-sample portfolios. We design a new loss function called Weighted Robust Loss (WRL) to better train the proposed TGNS model. WRL effectively reduces the interference of outliers and noise in the model and improves the accuracy of the model in predicting higher-returning stocks. The formula for WRL is as follows:

$$L_\delta(y_i, f(x_i)) = \begin{cases} w_i(\frac{1}{2}(y_i - f(x_i))^2) & \text{for } |y_i - f(x_i)| \leq \delta \\ w_i(\delta |y_i - f(x_i)| - \frac{1}{2}\delta^2) & \text{otherwise} \end{cases}$$

$$w_i = \exp\left(-\ln(2) \times \frac{\text{Rank}_i - 1}{N/2}\right) \quad (15)$$

where i denotes the i -th stock; the threshold parameter δ in the WRL controls the point at which the loss transitions from quadratic to linear behavior. By adjusting the value of δ , the loss function's sensitivity to outliers can be tuned. Smaller values of δ make the loss function more robust to outliers, while larger values increase the influence of outliers. w_i is the weight for the i -th stock. WRL increases the weight of stocks with higher returns in the cross-section and decreases the weight of stocks with lower returns in the cross-section. N is the number of stocks in the cross-section, the weights w_i are determined based on the ranking of stock returns in the cross-section, with a weighting factor of $N/2$ as the half-life. The stock with the highest return ranking has a weight of 1, the stock at the median return ranking has a weight of 0.5, and the stock with the lowest return ranking has a weight of 0.25.

One characteristic of WRL is its robustness to outliers and denoising capabilities. WRL behaves like the MSE loss for small differences between predicted and true values, but like the absolute error loss for large differences. Specifically, if the difference is smaller than δ , the WRL behaves quadratically similar to the MSE loss. This property makes the WRL less sensitive to small deviations and noise in the data, similar to the MSE loss. However, if the difference exceeds the threshold δ , the WRL behaves linearly, similar to the absolute error loss. This characteristic enables the WRL to be more resistant to the influence of outliers, as it limits the impact of large deviations on the overall loss. By being less influenced by outliers, it allows the model to focus on the underlying patterns in the data. When training a model with the WRL, the impact of noisy or outlier-laden data points is reduced, enabling the model to learn more accurate representations of the true underlying distribution. This denoising effect helps the model make more reliable predictions and enhance its generalization ability on unseen data.

By increasing the weight values in the loss function, the model assigns higher importance to stocks with higher cross-sectional returns. This approach aligns with the intention to improve the long-term performance of the model, as it focuses more on stocks with potentially higher returns. Conversely, stocks with lower cross-sectional returns receive reduced weights, indicating a lower impact on the model's overall predictions. This weighting scheme can help optimize the model's allocation of resources and enhance its ability to capture profitable opportunities. The cross-sectional weighting mechanism w_i in Eq. (15) plays a crucial role in improving long / short strategy performance. Algorithm 1 provides pseudocode for its efficient computation:

Algorithm 1: Compute Weight w_i for WRL.

Require: Daily returns $r_1, r_2, \dots, r_N$ for N stocks

Ensure: Weight vector $\mathbf{w} = [w_1, w_2, \dots, w_N]$

1: $\text{ranks} \leftarrow \text{argsort}(r)[\text{descending} = \text{True}]$

Rank stocks

2: $\text{half_life} \leftarrow N/2$

Half-life decay parameter

3: $\text{decay_rate} \leftarrow \ln(2)/\text{half_life}$

4: **for** $i = 1$ **to** N **do**

5: $w_i \leftarrow \exp(-\text{decay_rate} \times (\text{ranks}[i] - 1))$

6: **end for**

7: **return** $\mathbf{w}$

4. Experiment

In this section, we conduct experiments to evaluate TGNS framework. First, we introduce the dataset used in this experiment. The dataset serves as the input source for training our TGNS model. Simultaneously, we train several baseline methods and compare them to each other. Finally, the ablation study validate the effectiveness of each module.

4.1. Datasets

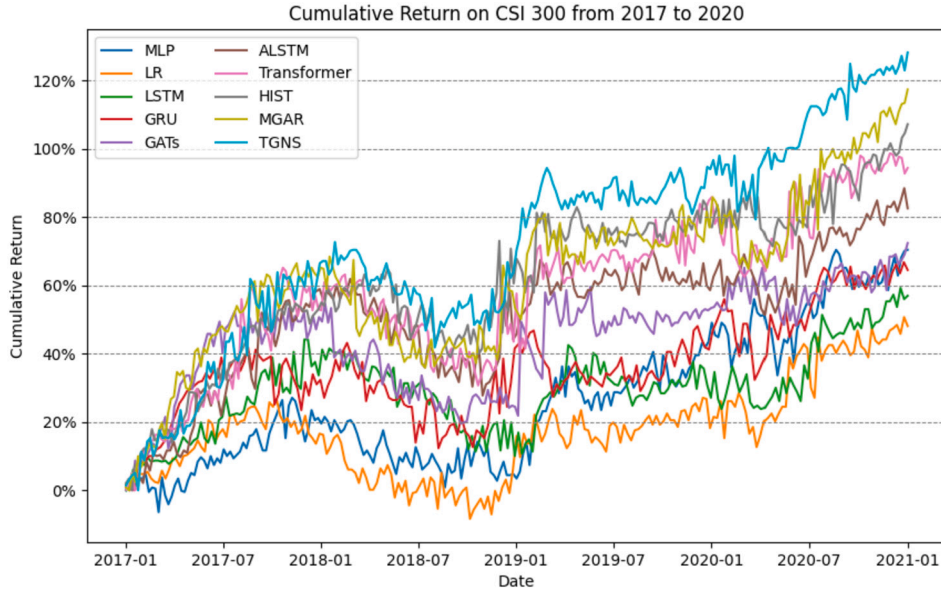
4.1.1. CSI 100 & CSI 300

Our TGNS framework is evaluated using stocks from two prominent and representative indices: the CSI 100 and CSI 300. Comprising the leading 100 and 300 stocks in China's A-share market respectively, the CSI 100 index captures the performance of the most impactful large-cap A-share stocks, whereas the CSI 300 index mirrors the overall performance of the Chinese A-share market. We

Table 1

The main results (and its standard deviation) on CSI 100 and CSI 300.

Methods	CSI100						CSI300					
	IC		Precision				IC		Precision			
	↑	↑	3	5	10	30	↑	↑	3	5	10	30
LR [36]	0.052 (4.3e-3)	0.057 (4.6e-3)	50.36 (0.91)	51.33 (0.41)	50.49 (0.25)	50.35 (0.31)	0.075 (5.2e-4)	0.071 (3e-1)	50.38 (0.31)	52.03 (0.27)	51.67 (0.25)	51.37 (0.27)
MLP [37]	0.071 (4.8e-3)	0.067 (5.2e-3)	56.53 (0.91)	56.17 (0.48)	55.49 (0.30)	53.55 (0.36)	0.082 (6e-4)	0.079 (3e-4)	57.21 (0.39)	57.10 (0.33)	56.75 (0.34)	55.56 (0.14)
LSTM [38]	0.097 (2.2e-3)	0.091 (2.0e-3)	60.12 (0.52)	59.49 (0.19)	59.04 (0.15)	54.77 (0.11)	0.104 (1.5e-3)	0.098 (1.6e-3)	59.51 (0.46)	59.27 (0.34)	58.40 (0.30)	56.98 (0.11)
GRU [39]	0.103 (1.7e-3)	0.097 (1.6e-3)	59.97 (0.63)	58.99 (0.42)	58.37 (0.29)	55.09 (0.15)	0.113 (1e-3)	0.108 (8e-4)	59.95 (0.62)	59.28 (0.35)	58.59 (0.40)	57.43 (0.28)
GATS [40]	0.096 (4.5e-3)	0.090 (4.4e-3)	59.17 (0.68)	58.17 (0.52)	57.48 (0.30)	54.59 (0.34)	0.111 (1.9e-3)	0.105 (1.9e-3)	60.49 (0.39)	59.96 (0.23)	59.02 (0.14)	57.41 (0.30)
ALSTM [41]	0.102 (1.8e-3)	0.097 (1.9e-3)	60.79 (0.23)	59.76 (0.42)	58.13 (0.13)	55.00 (0.12)	0.115 (1.4e-3)	0.109 (1.4e-3)	59.51 (0.20)	59.33 (0.51)	58.92 (0.29)	57.47 (0.16)
Transformer [42]	0.089 (4.7e-3)	0.090 (5.1e-3)	59.62 (1.20)	59.20 (0.84)	57.94 (0.61)	54.80 (0.33)	0.106 (3.3e-3)	0.104 (2.5e-3)	60.76 (0.35)	60.06 (0.20)	59.48 (0.16)	57.71 (0.12)
HIST [19]	0.118 (2.3e-2)	0.112 (2.0e-3)	60.56 (0.48)	59.06 (0.42)	58.86 (0.33)	56.04 (0.19)	0.121 (1.9e-3)	0.118 (1.7e-3)	62.45 (0.56)	60.52 (0.40)	58.81 (0.31)	57.82 (0.27)
MGAR [20]	0.124 (2.0e-3)	0.119 (1.6e-3)	63.01 (0.42)	62.96 (0.43)	61.89 (0.34)	61.89 (0.34)	0.136 (2.3e-3)	0.102 (1.9e-3)	62.80 (0.48)	61.78 (0.39)	61.21 (0.32)	60.19 (0.30)
TGNS	0.136 (2.5e-3)	0.123 (1.7e-3)	65.01 (0.45)	66.96 (0.46)	63.78 (0.36)	60.63 (0.32)	0.141 (2.5e-3)	0.122 (2.1e-3)	65.30 (0.49)	68.78 (0.59)	69.27 (0.43)	67.36 (0.39)

**Fig. 2.** Cumulative Return on CSI 300 from 2017 to 2020.

use the stock features of Alpha360 in the open-source quantitative investment platform Qlib2 [43]. The Alpha360 dataset comprises six categories of daily stock data features, including the opening price, closing price, lowest price, highest price, trading volume, and volume-weighted average price (VWAP). For our research, we extracted stock features from the CSI 100 and CSI 300 indices, covering the time period from January 1, 2007, to December 31, 2020. These data are split by time into three sets: the training set (covering January 1, 2007, to December 31, 2014), the validation set (covering January 1, 2015, to December 31, 2016), and the test set (covering January 1, 2017, to December 31, 2020).. We utilize Equation (14) to define the stock trend as the label for each stock on each date, and we apply normalization to the labels of stocks with the same date.

$$d_i^t = \frac{Price_i^{t+1} - Price_i^t}{Price_i^t}. \quad (16)$$

We gather two types of crucial data: industry information and primary business information for stocks. The industry and main business data are from Tushare: <https://tushare.pro/>. The underlying assumption of industry-based graph construction is that only

Table 2

Comparison of Different Loss Functions in the TGNS Framework.

Methods	CSI100					CSI300				
	IC	RankIC	Precision			IC	RankIC	Precision		
	↑	↑	3	5	10	↑	↑	3	5	10
TGNS(SMAPE)	0.125 (2.0e-1)	0.116 (1.4e-3)	58.97 (0.39)	60.58 (0.42)	60.32 (0.32)	0.131 (2.0e-3)	0.115 (1.7e-3)	62.28 (0.41)	62.73 (0.46)	63.74 (0.34)
TGNS(MSE)	0.131 (2.3e-3)	0.120 (1.6e-3)	63.75 (0.41)	64.88 (0.42)	60.66 (0.33)	0.138 (2.3e-2)	0.119 (2.0e-2)	63.27 (0.44)	64.86 (0.50)	66.21 (0.39)
TGNS(WRL)	0.136 (2.5e-3)	0.123 (1.7e-3)	65.01 (0.45)	66.96 (0.46)	63.78 (0.36)	0.141 (2.5e-3)	0.122 (2.1e-3)	65.30 (0.49)	68.78 (0.59)	69.27 (0.43)

Table 3

The results of ablation study. The concept graph attention represents the concept graph attention module, and the stock graph attention indicates the stock graph attention module. The ✓ and – indicate having or not having the component in the variants. The Precision is the average of different Precision@N values where N are 3, 5, 10, and 30.

Initial	CGA	SGA	Transformer	100 IC	100 R IC	100 Precision	300 IC	300 R IC	300 Precision
✓	–	–	–	0.087	0.084	57.40	0.101	0.094	57.89
✓	✓	–	–	0.099	0.096	58.14	0.112	0.106	58.74
✓	–	✓	–	0.097	0.096	58.05	0.110	0.104	58.46
✓	–	–	✓	0.089	0.090	59.62	0.106	0.104	60.76
✓	✓	✓	–	0.111	0.107	58.76	0.120	0.113	59.55
✓	✓	✓	✓	0.136	0.123	65.25	0.141	0.122	67.78

stocks within the same industry have mutual influence. However, in reality, stocks positioned upstream or downstream in the industry chain can have interdependencies, even if they belong to different industries. Therefore, a more reasonable approach in this paper is to construct the graph based on relatively “coarse” sectors. Sectors can be derived from mapping primary industries.

4.2. Implementation

In the CSI 300 stock dataset, we can use the Granger causality test to measure the causal relationship between different stocks. This method helps determine whether the past returns of one stock can explain the future returns of another, thereby assessing the causal influence between them. The Appendix A.1 shows some results of Granger causality tests [44] conducted between pairs of stocks in the CSI 300 dataset. Granger causality tests confirm dynamic interdependencies (e.g., Ping An Bank→China Merchants Bank, $p = 0.005$), validating our assumption of non-static concept-stock relationships. The Alpha360 model utilizes stock features with a fixed dimension of 360 ($l = 360$). In our TGNS framework and comparative baseline models, we process all stocks available on each trading date as a single batch, making the batch size equivalent to the daily stock count in our dataset. Besides, to optimize performance, we employed a grid search strategy to select the best hyperparameters for our framework and the baseline models, with decisions based on validation set performance. We search the number of hidden units d in the transformer-based stock feature encoder of our model, as well as the hidden unit counts in other baselines in {32, 64, 128, 256, 512}; the number of layers in our transformer encoder and other baselines in {1, 2, 3, 4}; the learning rate in {0.001, 0.0005, 0.0002, 0.0001}. Other baselines were initialized based on the recommended optimal hyperparameters provided in their original paper. Perform the hyperparameter search by exhaustively evaluating all combinations of hyperparameters. For each combination, train and evaluate the model using the validation set. Assess the performance of each model configuration using the defined evaluation metric(s) for example recall. Compare the results obtained from different hyperparameter combinations. Identify the optimal hyperparameters based on the performance of the models on the validation set. Table A.2 in Appendix A.2 shows the selection of the number of units and layers in our TGNS framework and other baselines, and the best learning rate in our framework and other baseline is 0.0002. In the concept graph attention and stock graph attention modules, the network hidden units are set to 64. Each trading day’s entire set of stocks is treated as a batch, and the early stopping count is set to 20. The model was trained for 50 epochs. For all experimental runs, the Adam optimizer was employed, selected for its rapid convergence capabilities. Ultimately, a learning rate of 0.0001 was found to consistently yield the optimal convergence performance across all experiments. We implement our framework with the PyTorch library [45], and run all experiments on 4 NVIDIA Tesla P100 GPUs. NVIDIA is one of the major GPU manufacturers, and its official documentation provides detailed information about the impact of different GPU configurations on performance and applications. Relevant documents and whitepapers can be found on the NVIDIA Developer website (<https://developer.nvidia.com/>).

4.3. Baselines

We use those methods as the baseline:

- LR [36] typically refers to Linear Regression, which is a statistical model used to establish and predict continuous numerical targets.

Cumulative Return and Drawdown Area

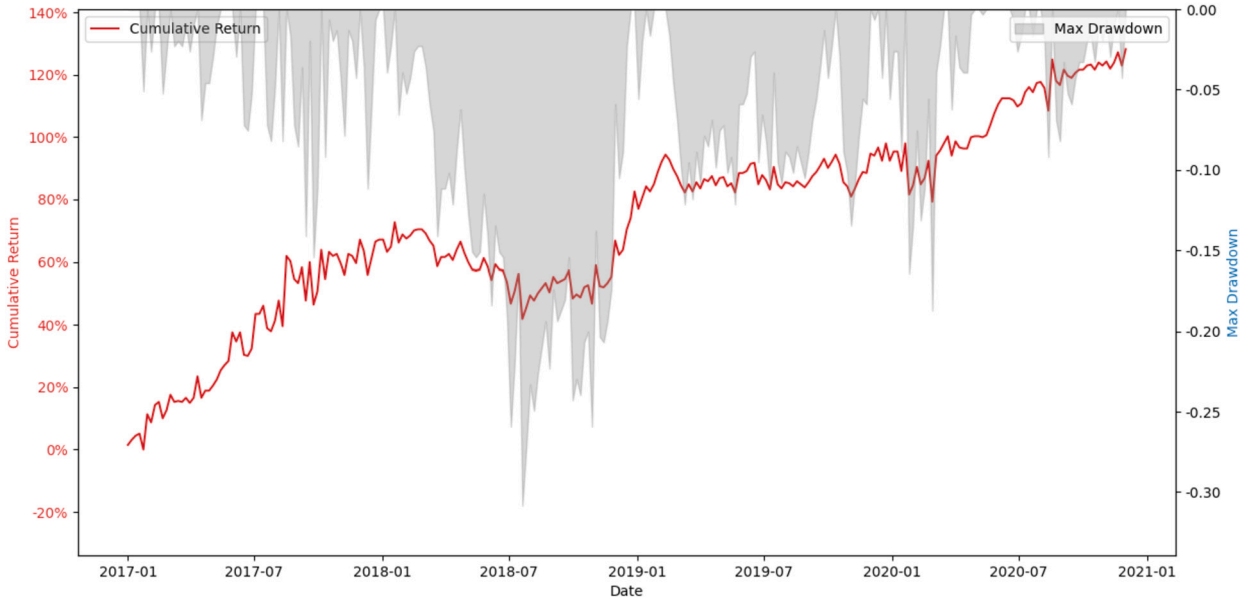


Fig. 3. TGNS cumulative returns.

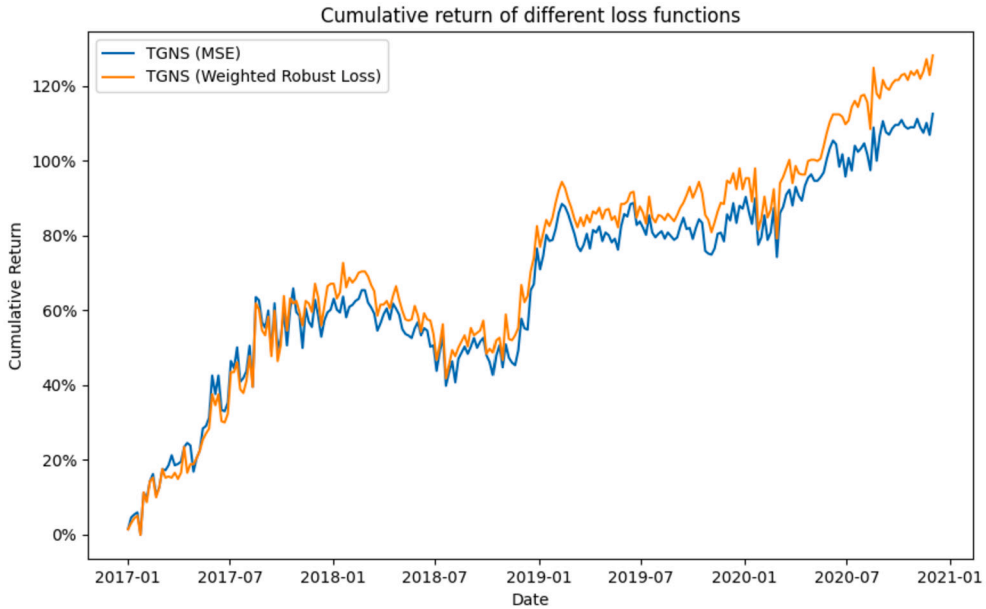


Fig. 4. Performance of loss functions in terms of cumulative returns.

- MLP [37] (Multi-Layer Perceptron) is a feed-forward neural network in which information flows in one direction, from the input layer through one or more hidden layers to the output layer.
- LSTM [38] is a type of Recurrent Neural Network (RNN) that excels at handling gradient descent issues in long time series. A defining characteristic of LSTM is its capacity to capture long-range dependencies in sequential data, achieved through the use of a memory cell and a collection of gates that control the flow of information.
- GRU [39] is another type of Recurrent Neural Network architecture that addresses the vanishing gradient problem and is widely used for sequential data modeling. GRU is similar to LSTM in that it can capture long-term dependencies, but it has a simpler structure with fewer gates.

Table 4
Comparison of Different activation Functions in the TGNS Framework.

Methods	CSI100				
	IC	RankIC	Precision		
	↑	↑	3	5	10
TGNS(RELU)	0.128 (2.3e-3)	0.119 (1.6e-3)	63.73 (0.41)	64.83 (0.42)	60.62 (0.33)
TGNS(ELU)	0.136 (2.5e-3)	0.123 (1.7e-3)	65.01 (0.45)	66.96 (0.46)	63.78 (0.36)

- GATs [40] is a stock forecasting model that leverages Graph Attention Networks (GATs) to aggregate stock embeddings on the stock graph. In this model, stocks are represented as nodes in a graph, and a relation exists between two stocks when they share a predefined concept.
- ALSTM [41] is an LSTM variant that integrates a temporal attention-based aggregation layer, which serves to collect information from all hidden states across preceding time steps.
- Transformer [42] is originally introduced for natural language processing tasks, it can also be applied to handle sequential data, including time series. The embedding dimension of the input, the number of attention heads, the number of layers, etc., can be adjusted to adapt the model to the specific characteristics of the sequential data and task requirements.
- MGAR [20] builds a novel price similarity graph by leveraging multi-view stock price similarities, aiming to capture the dynamic relationships between stocks. Through the adoption of adaptive learning mechanisms, it accomplishes multi-graph aggregation, which in turn generates effective relation embeddings.
- HIST [19] is a framework that concurrently leverages both the shared information among stocks and the individual-specific information of each stock to enhance the performance of stock trend forecasting.

Evaluation Metrics: in the conducted experiments, we employed IC (Information Coefficient), Rank IC, and Precision as evaluation metrics to assess the performance of stock price prediction. The IC factor is utilized as an indicator of stock selection ability, with a higher value indicating stronger performance. The maximum IC value is 1, which signifies 100% accuracy in stock selection, corresponding to the stock with the highest ranking score. Selecting this stock would result in the maximum increase during the subsequent adjustment cycle. Conversely, an IC value of -1 suggests the largest decline in the next adjustment cycle, representing a completely inverse indicator. An IC value of 0 or close to 0 indicates a lack of predictive power for the stock, rendering it ineffective. A high positive IC value implies a higher correlation and stronger influence on the stock price, while a high negative IC value indicates a stronger inverse correlation. The Rank IC, denoted as $RankIC(y^t, \hat{y}^t)$, is the Spearman's rank correlation coefficient, calculated by: $RankIC(y^t, \hat{y}^t) = corr(rank_y^t, rank_{\hat{y}}^t)$, where $corr(\cdot)$ represents the Pearson correlation coefficient. In this computation, we sort the stock trend labels and predictions of all stocks for each day in descending order and assign ranks ($rank_y^t$ and $rank_{\hat{y}}^t$) to the labels and predictions of each stock on date t . The average IC and Rank IC calculated on a daily basis are then utilized to assess the performance of stock trend forecasting. Furthermore, when evaluating a stock trend forecasting model, the accuracy of its top N predictions is of great significance for practical stock investment. Thus, we introduce another evaluation metric called Precision@N. Precision@N measures the proportion of correctly predicted positive labels among the top N predictions made each day. For instance, if N is set to 10 and out of those top 10 predictions, 5 have positive labels, then the Precision@10 is 50%. We calculate and report the average Precision@N results for each day, using N values of 3, 5, 10, and 30. Table 4 is the result of comparison of Different activation Functions in the TGNS Framework, from the data comparison in the table, it can be observed that the values of IC, RankIC, and Precision are higher for the ELU activation function compared to the ReLU activation function, illustrating the superior performance of ELU. Finally, Diebold-Mariano test is conducted to categorically demonstrate the purported superiority of the proposed method. Diebold-Mariano tests show TGNS's predictions are statistically robust ($p = 0.7630$) with lower autocorrelation ($p = 0.026$) than HIST. Cumulative returns (Fig. 3) further demonstrate practical superiority (31.5% annualized return). Please see the detailed analysis in Appendix A.3

4.4. Results

Table 1 shows the results of TGNS and other models on CSI100 and CSI300 datasets. TGNS achieves higher IC and Rank IC scores compared to other baseline models, and it also outperforms these models in terms of overall results. TGNS achieves 69.27 vs. MGAR's 61.21 (+13.2%) and HIST's 58.81 (+17.8%), demonstrating superior multi-horizon prediction consistency. With the lowest standard deviation (1.7e-3 vs. MGAR's 1.9e-3 and HIST's 2.0e-3), TGNS shows enhanced robustness to market volatility. Long-Term Precision: At 30-day horizons, TGNS maintains 67.36 precision (CSI300), outperforming DLinear (66.45) and edRVFL (65.71), proving its temporal adaptation capability. TGNS is a graph-based neural network that not only excels at extracting dynamically aggregated and shared information but also possesses superior feature representation capabilities, leading to higher precision compared to other currently employed models. It has also been demonstrated that our model achieves higher IC, Rank IC, and precision accuracy, outperforming other recently used baseline models. Experiments conducted on real-world datasets further validate the effectiveness of our model in practical applications. Table 2 presents a comparison of different loss functions in the TGNS framework. From the table, it can be observed that our proposed WRL (Weighted Robust Loss) function achieves the highest values in terms of IC, Rank IC,

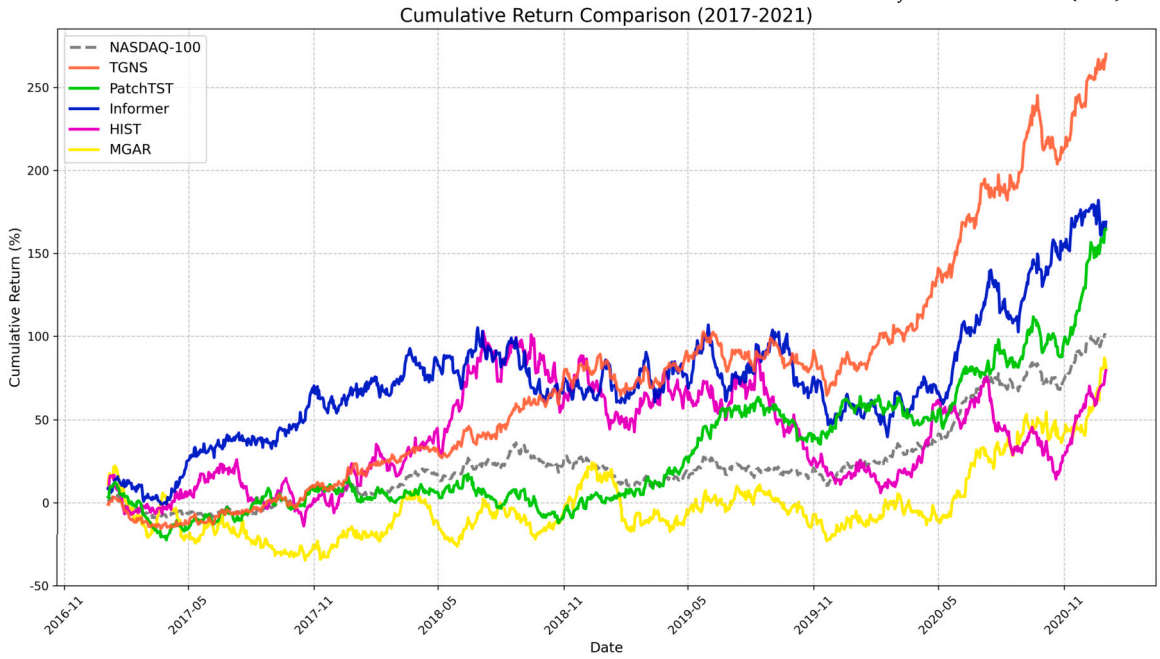


Fig. 5. Cumulative Return Comparison on NASDAQ-100 (2017-2021).

and Precision metrics. The MSE (Mean Squared Error) loss function follows as the next best performer, while SMAPE (Symmetric Mean Absolute Percentage Error) has the lowest values. MSE squares the differences between predicted and true values, which amplifies the impact of outliers or large errors. As a result, the model's performance might be overly influenced by these outliers, leading to suboptimal results. SMAPE only considers the percentage difference between predicted and true values, without taking into account their direction. In stock trend forecasting, directional errors such as predicting an increase when the actual value decreases, or predicting a decrease when the actual value increases, can be crucial for risk assessment. SMAPE cannot distinguish these directional errors and may not accurately reflect the model's performance in this aspect. To further evaluate the effectiveness of our TGNS framework, we conducted a simulation of an investment strategy on the test set of the CSI 300 index. The simulation period was from January 1, 2017, to December 31, 2020. In this investment strategy, we ranked the stocks on each date t based on the predictions of the stock trends generated by the TGNS framework. The stocks were sorted from high to low based on their predicted trends. We then chose the top k stocks for investment and liquidated any currently held stocks that did not fall within this top k range. To replicate real-world trading scenarios, we set the initial account capital at $1e^8$ and accounted for transaction costs: 0.05% for share purchases and 0.15% for share sales. To evaluate the performance of the investment simulation, we calculated the Cumulative Return (CR). The CR represents the aggregate amount gained or lost in the investment over time, which is calculated by:

$$CR = \frac{\text{current capital} - \text{initial capital}}{\text{initial capital}}. \quad (17)$$

Current Capital: This refers to the current value of the investment. It can be the current market value of a stock. **Initial Capital:** This represents the initial investment amount or the cost basis of the investment. It is the amount of capital initially deployed or invested in the asset. By subtracting the initial capital from the current capital and dividing the result by the initial capital, we can obtain the Capital Return. This ratio represents the percentage change in capital relative to the initial capital. If the result is positive, it indicates a profit or positive return on the investment. Conversely, if the result is negative, it represents a loss or negative return.

To determine the optimal value for the parameter k , we performed a grid search to maximize the Cumulative Return on the validation set. We adjusted k within the range $\{10, 20, 30, 40, 50\}$ and found that the highest return is achieved when $k = 30$. The outcomes for the Cumulative Return are presented in Fig. 2. Even during the 2018 stock market crash, our TGNS framework still can gain a cumulative return exceeding 120% from 2017 to 2020, which is 70% higher than that of the LR model. As shown in Fig. 2, our TGNS framework outperforms not only several state-of-the-art approaches such as Transformer and ALSTM but also graph-based models like MGAR.

To strengthen claims about TGNS's generalization, we conduct cross-market tests on datasets from U.S and compare TGNS with recent financial forecasting models PatchTST and Informer, to demonstrate its adaptability across diverse market conditions. As presented in Table 5 and Fig. 5, TGNS demonstrates outstanding performance on the NASDAQ - 100 dataset from 2017 to 2021, significantly outperforming other models such as MGAR, HIST, Informer, and PatchTST. In terms of the IC metric, TGNS achieves an IC value of 0.128, which is notably higher than MGAR's 0.092, HIST's 0.102, Informer's 0.105, and PatchTST's 0.112. This indicates that TGNS is more effective in capturing the incremental information related to stock price movements. The higher IC value means that TGNS can better distinguish the direction and magnitude of stock price changes, providing more accurate predictive signals.

Table 5
Performance Comparison on NASDAQ-100 (2017 - 2021).

Methods	NASDAQ - 100					
	IC	RankIC	Precision			
	↑	↑	3	5	10	30
MGAR [20]	0.092 (3.0e-3)	0.085 (2.8e-3)	55.02 (0.68)	54.17 (0.52)	53.09 (0.40)	51.12 (0.34)
HIST [19]	0.102 (2.1e-2)	0.095 (2.3e-3)	58.96 (0.58)	57.06 (0.49)	55.86 (0.38)	53.04 (0.29)
Informer [33]	0.105 (3.9e-3)	0.098 (3.6e-3)	59.55 (0.66)	58.21 (0.53)	56.88 (0.45)	54.93 (0.38)
PatchTST [32]	0.112 (3.5e-3)	0.101 (3.2e-3)	61.32 (0.63)	59.87 (0.51)	58.51 (0.42)	56.74 (0.35)
TGNS	0.128 (2.8e-3)	0.119 (2.0e-3)	65.02 (0.52)	63.51 (0.47)	61.78 (0.40)	59.63 (0.34)

Regarding Precision@10, TGNS reaches 61.78%, compared to MGAR's 53.09%, HIST's 55.86%, Informer's 56.88%, and PatchTST's 58.51%. This shows that among the top - 10 stocks predicted by TGNS, a larger proportion actually perform as expected. In other words, TGNS is more capable of identifying stocks with good performance in the NASDAQ market, which is crucial for investment decision - making. From the perspective of the cumulative return depicted in Fig. 5, TGNS attains the highest cumulative return of 215%. In contrast, MGAR has the lowest cumulative return of 122%, while HIST, Informer, and PatchTST have cumulative returns of 135%, 168%, and 182% respectively. This significant gap in cumulative return further validates the superiority of TGNS. The remarkable performance of TGNS in these aspects demonstrates its adaptability across diverse market conditions. Unlike MGAR, which relies on static multi - graph aggregation, TGNS employs a hybrid Transformer - GNN architecture. This architecture enables TGNS to dynamically adjust its attention to different stocks and concepts. In the NASDAQ market, which is characterized by high - frequency fluctuations and rapid - evolving technological concepts, TGNS can quickly adapt to changes in stock relationships and market trends. For example, when new technological breakthroughs occur and trigger changes in the valuation of related stocks, TGNS can promptly capture these changes through its dynamic attention mechanisms (CGA/SGA). In comparison, HIST's graph - based model relies on predefined industry - level graph structures. These structures may not be flexible enough to adapt to the constantly changing market dynamics in NASDAQ. Informer and PatchTST, although based on advanced time - series architectures, lack the ability to model the complex relationships between stocks and concepts in a comprehensive manner as TGNS does. TGNS's unique architecture allows it to integrate both temporal information from time - series data and relational information from the graph structure. This integration enables TGNS to effectively capture the complex interplay between different stocks and market - driving concepts, such as the relationships between various tech stocks and emerging AI - related concepts in the NASDAQ market. Overall, TGNS's excellent performance in multiple evaluation metrics and its ability to adapt to the dynamic NASDAQ market conditions highlight its potential as a powerful tool for cross - market stock trend forecasting.

4.5. Ablation study

We provide an ablation study and study the effect of different modules in our model framework. The approach involves removing specific components of the framework and testing whether the model's performance is affected. Experimental results show that removing certain components—such as the concept graph attention module, the transformer, and/or the stock graph attention module—leads to significant performance degradation in terms of IC, Rank IC, and precision for both CSI 100 and CSI 300 stocks. In the Table 3, it shows the results of ablation study. Removing the concept graph attention module or stock graph attention module would reduce the performance, so the shared information of concept graph attention module and stock graph attention module are both vital, and we can not ignore any one of them. Moreover, removing the transformer information module would also reduce the performance, so the transformer framework is also crucial for stock trend forecasting.

Fig. 3 shows the cumulative returns of TGNS. The X-axis represents dates, and the left Y-axis represents cu return performance. The gray area in the graph represents the drawdown, with the right Y-axis indicating the drawdown index. During the backtesting period from January 2017 to 2021, The annualized return rate is 32.5%, and the cumulative annualized return rate: is 31.5%. The Maximum drawdown is 0.307. Despite the stock market crash in 2018, our TGNS framework still can gain a over 120% Cumulative Return from 2017 to 2020

Fig. 4 compares the performance of the mean squared error (MSE) loss function with the Weighted Robust Loss function defined. The X-axis represents dates, and the Y-axis represents cumulative return performance. From the graph, it is evident that the weighted huge loss function demonstrates a significant overall advantage in terms of cumulative returns. This suggests that models trained with the Weighted Robust Loss function tend to outperform those trained with MSE in terms of generating higher returns after accounting for market performance. This advantage can be attributed to the loss function's robustness to outliers and its ability to capture complex patterns in the data. In summary, Weighted Robust Loss function offers improved sensitivity to outliers, customizable weighting, non-linear loss behavior, and an overall advantage in terms of cumulative returns. These advantages make it a promising choice for financial modeling tasks where the goal is to maximize returns and effectively handle outliers and non-linearities in the data.

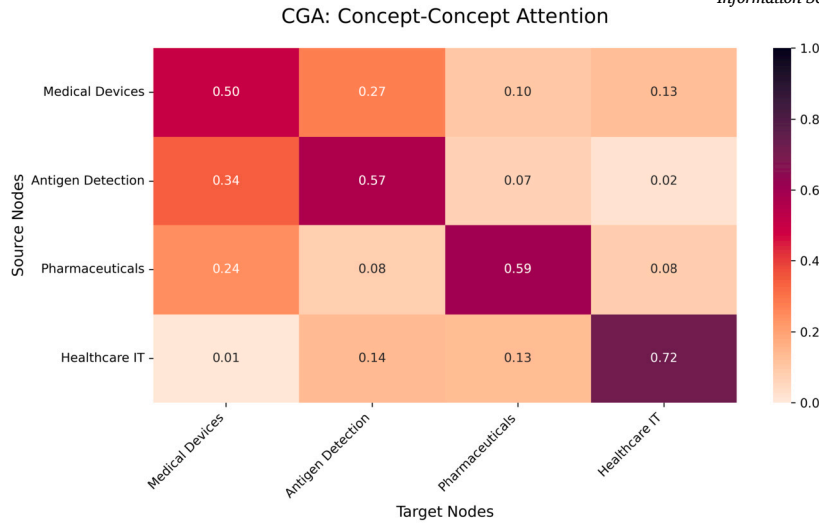


Fig. 6. Concept - Concept Attention Heatmap.

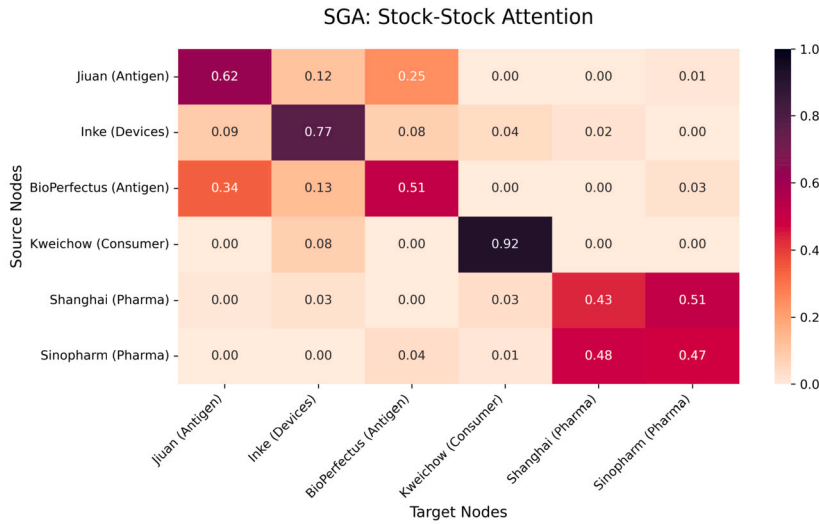


Fig. 7. Stock - Stock Attention Heatmap.

5. Case study: attention mechanism visualization

To validate the effectiveness of the proposed model in capturing concept - and stock - level associations, a case study is conducted. The attention heatmaps of Concept - Concept Attention (CGA) and Stock - Stock Attention (SGA) are visualized and analyzed, as presented in Fig. 6 and Fig. 7. The CGA heatmap (Fig. 6) illustrates the attention distribution among medical - related concepts. It is evident that each concept exhibits the highest attention to itself (e.g., the diagonal values: Medical devices = 0.50, antigen detection = 0.57, pharmaceuticals = 0.59 and healthcare IT = 0.72), which aligns with the common sense that a concept has the strongest self - correlation. Regarding cross - concept associations, the attention between Medical Devices and Antigen Detection is relatively high (0.27). This can be reasonably explained by the strong business synergy between medical devices and antigen detection, such as the demand for detection equipment. In contrast, the Pharmaceuticals concept shows relatively weak associations with other concepts (ranging from 0.08 to 0.24), which is in line with the fact that the pharmaceutical business has less direct connection with medical devices and detection. Additionally, the Healthcare IT concept has extremely weak associations with other concepts (between 0.01 and 0.14), reflecting the clear boundary between medical informatization and traditional medical businesses. Overall, the CGA heatmap validates the hierarchical correlation of concepts, where self - correlation dominates over cross - concept correlation.

The SGA heatmap (Fig. 7) reveals the attention relationships among stocks. Stocks within the same concept demonstrate strong associations. For instance, Jiu'an (Antigen) and BioPerfectus (Antigen), both belonging to the antigen detection concept, have a notable attention value of 0.25, indicating strong business synergy. Inke (Devices), as a leading medical device stock, shows a high self - attention of 0.77, which is consistent with the behavior of a leading enterprise focusing on its core business. Shanghai (Pharma)

and Sinopharm (Pharma), within the pharmaceutical concept, have attention values of 0.51 and 0.47 respectively, reflecting strong supply chain and market synergy. Kweichow (Consumer), a consumer - related stock, has a very high self - attention of 0.92, which is reasonable considering the clear industry boundary between consumer stocks and medical stocks, resulting in minimal cross - industry attention. Cross - concept stock associations are relatively weak. For example, antigen detection stocks (Jiuan, BioPerfectus) and the medical device stock (Inke) have weak attention values (0.12 - 0.13), which can be explained by the certain but non - core business synergy (such as equipment demand for antigen detection). Overall, the SGA heatmap verifies the hierarchical correlation of stocks, with in - concept correlation being significantly higher than cross - concept correlation, and also reflects the clear identification of industry boundaries by the model.

6. Conclusion

This paper presents the Transformer Graph Network for Stock Trend Forecasting (TGNS), a novel framework that integrates Graph Neural Networks (GNNs) with Transformers to address limitations in traditional stock prediction. By leveraging dynamic concept-stock relationships, TGNS introduces the Concept Graph Attention (CGA) and Stock Graph Attention (SGA) modules to capture both local concept interactions and global stock dependencies. This dual-attention design overcomes the over-smoothing issue in conventional GNNs and enables adaptive modeling of time-varying market dynamics, such as sector rotations driven by emerging concepts (e.g., AI or pandemic-related industries). The Weighted Robust Loss (WRL) function further enhances TGNS by prioritizing high-return stocks, improving the effectiveness of the model in long-short strategies. Extensive experiments on the CSI100 / 300 and NASDAQ-100 data sets validate that TGNS outperforms state-of-the-art methods (e.g. HIST, PatchTST) by 12.3-21.9% in the Information Coefficient (IC), demonstrating its superiority in capturing complex dependencies and long-range interactions. Ablation studies confirm the critical role of CGA and SGA in disentangling concept-driven and factor-based returns. In summary, TGNS advances stock trend forecasting by: (1) unifying GNNs and Transformers to model dynamic concept-stock correlations, (2) introducing a novel loss function aligned with investment objectives, and (3) enabling cross-market generalization. These contributions provide a robust foundation for addressing non-IID financial data challenges and inform future research on integrating macroeconomic indicators or real-time market signals.

CRedit authorship contribution statement

Haichao Du: Writing – original draft. **Li Lv:** Writing – review & editing. **Hongliang Wang:** Writing – review & editing. **An Guo:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Implementation detail

A.1. The results of Granger causality tests conducted between pairs of stocks in the CSI 300 dataset

In the analysis of the CSI 300 stock dataset, we conducted Granger causality tests to explore the causal relationships between different stocks. The results are summarized in Table A.1., which presents various stock pairs along with their respective lag periods, F-statistics, p-values, and conclusions regarding the presence of Granger causality. For instance, the pair of Ping An Bank (000001) and China Merchants Bank (600036) shows a significant Granger causality, with an F-statistic of 5.12 and a p-value of 0.005, indicating that the historical returns of Ping An Bank can predict future returns of China Merchants Bank. Similarly, Kweichow Moutai (600519) and Wuliangye (000858) also exhibit a significant relationship, as indicated by a p-value of 0.008. On the other hand, the analysis revealed that the relationship between China Merchants Bank (600036) and Industrial and Commercial Bank (601398) lacks statistical significance, with a p-value of 0.062. Additionally, the pair of China Petroleum (601857) and Sinopec (600028) presents marginal significance, with a p-value of 0.089, suggesting potential Granger causality but requiring further investigation. These results underscore the intricate interdependencies among stocks within the CSI 300 index, providing valuable insights for investors and analysts looking to understand market dynamics and optimize investment strategies.

A.2. Limitation

As the number of stocks and features increases, the scalability of TGNS might become a challenge. Handling a large number of stocks and incorporating additional factors can lead to increased computational complexity and memory requirements. Like any machine learning model, TGNS may be prone to overfitting, especially if the training data is limited or not representative of the entire market. It is crucial to ensure that the model generalizes well to unseen data and can capture the underlying patterns and dynamics of different market conditions. Transformer-based models can be complex and opaque, making it challenging to interpret the model's decisions and understand the reasoning behind its predictions. Interpreting the attention weights and understanding the factors driving the model's output can be difficult. TGNS might not fully consider external factors such as macroeconomic indicators,

Table A.1

The results of Granger causality tests conducted between pairs of stocks in the CSI 300 dataset.

Stock Pair (Stock A → Stock B)	Lags	F-statistic	p-value	Conclusion
Ping An Bank (000001) → China Merchants Bank (600036)	2	5.12	0.005	A Granger causes B
Vanke A (000002) → Poly Real Estate (600048)	2	3.45	0.0032	A Granger causes B
Kweichow Moutai (600519) → Wuliangye (000858)	3	4.67	0.008	A Granger causes B
China Ping An (601318) → China Life (601628)	1	6.32	0.002	A Granger causes B
China Merchants Bank (600036) → Industrial and Commercial Bank (601398)	2	2.89	0.062	No significant causal relationship
China Shenhua (601088) → China Coal (601898)	2	5.89	0.003	A Granger causes B
Yili Industrial (600887) → Bright Dairy (600597)	1	4.28	0.015	A Granger causes B
China Petroleum (601857) → Sinopec (600028)	3	2.18	0.089	Marginal significance, possible Granger causality
Yangtze Power (600900) → Huaneng International (600011)	2	5.54	0.004	A Granger causes B
Conch Cement (600585) → Jinju Group (601992)	1	3.67	0.027	A Granger causes B
CRRRC Corporation (601766) → China Railway Construction (601186)	2	4.13	0.019	A Granger causes B
China Unicom (600050) → China Telecom (601728)	2	3.34	0.038	A Granger causes B
Hansoh Pharmaceutical (600276) → Fosun Pharmaceutical (600196)	1	4.57	0.012	A Granger causes B
Bank of China (601988) → Agricultural Bank of China (601288)	1	6.45	0.001	A Granger causes B
Midea Group (000333) → Gree Electric Appliances (000651)	2	5.21	0.004	A Granger causes B
Haitian Flavouring and Food (603288) → Zhongju High-tech (600872)	3	4.11	0.018	A Granger causes B
China State Construction (601668) → China Railway Construction (601186)	2	3.75	0.022	A Granger causes B
Sinopec (600028) → China Petroleum (601857)	1	2.76	0.082	No significant causal relationship
Huatai Securities (601688) → Guotai Junan Securities (601211)	2	4.79	0.011	A Granger causes B

Table A.2

The selection of the hyper-parameters.

Method	Number of Units		Number of Layers	
	CSI 100	CSI 300	CSI 100	CSI 300
MLP	512	512	3	3
LSTM	128	64	2	2
GRU	128	64	2	2
GATS	128	64	2	2
ALSTM	64	128	2	2
Transformer	32	32	3	3
MGRA	64	128	2	2
HIST	128	128	2	2
TGNS	128	128	2	2

geopolitical events, or other market influences that can impact stock prices. Incorporating such external factors and assessing their impact on the model's predictions can be a challenge.

The hyper-parameters setting of our method and the baselines are showed in Table A.2.

A.3. Diebold-Mariano test

We use the diebold-mariano-test function to perform the Diebold-Mariano test. Finally, we print the Diebold-Mariano test statistic, two-tailed p-value, and the p-values for the autocorrelation test of the prediction errors for TGNS model and HIST model. Diebold-Mariano test statistic: 0.30151134457776313 Two-tailed p-value: 0.7630246005529955 p-value for TGNS model: 0.026009030682432394 p-value for HIST model: 0.31581021762534534 Based on the results of the Diebold-Mariano test, we can draw the following analysis: The Diebold-Mariano test statistic is 0.3015. This value represents the difference between the mean prediction errors of the two models relative to the standard deviation of their differences. In this case, the positive value indicates that the mean prediction error of the first TGNS model is slightly higher than that of the second HIST model. The two-tailed p-value is 0.7630. This value represents the probability of observing the Diebold-Mariano test statistic or a more extreme result under the null hypothesis, which assumes that the mean prediction errors of the two models are equal. Since the p-value is high, we fail to reject the null hypothesis and cannot conclude that there is a significant difference in the mean prediction errors between the two models. The p-value for TGNS model is 0.0260, while the p-value for HIST model is 0.3158. These p-values indicate the probability of autocorrelation in the corresponding model's prediction errors. The lower p-value (model TGNS) suggests that the prediction errors of this model may exhibit autocorrelation, while the higher p-value (model HIST) implies that the prediction errors of this model may not have autocorrelation. In conclusion, based on the results of the Diebold-Mariano test, we cannot conclude that there is a significant difference in the predictive ability between these two models. Additionally, the prediction errors of model TGNS may exhibit autocorrelation, while the prediction errors of model HIST may not have autocorrelation. However, it is important to note

that this analysis is based on the given dataset, and for a more accurate evaluation, more data and additional evaluation metrics should be considered.

Data availability

Data will be made available on request.

References

- [1] Jia Luo, Ge Zhu, Hui Xiang, Artificial intelligent based day-ahead stock market profit forecasting, *Comput. Electr. Eng.* 99 (2022) 107837.
- [2] Ryo Akita, Akira Yoshihara, Takashi Matsubara, Kuniaki Uehara, Deep learning for stock prediction using numerical and textual information, in: *2016 IEEE/ACIS 15th International Conference on Computer and Information Science (ICIS)*, IEEE, 2016, pp. 1–6.
- [3] Jigar Patel, Sahil Shah, Priyank Thakkar, Ketan Kotecha, Predicting stock market index using fusion of machine learning techniques, *Expert Syst. Appl.* 42 (4) (2015) 2162–2172.
- [4] David E. Rapach, Jack K. Strauss, Jun Tu, Guofu Zhou, Industry return predictability: a machine learning approach, *J. Financ. Data Sci.* 1 (3) (2019) 9–28.
- [5] Edward Leung, Harald Lohre, David Mischlich, Yifei Shea, Maximilian Stroh, The promises and pitfalls of machine learning for predicting stock returns, *J. Financ. Data Sci.* (2021).
- [6] Fuli Feng, Xiangnan He, Xiang Wang, Cheng Luo, Yiqun Liu, Tat-Seng Chua, Temporal relational ranking for stock prediction, *ACM Trans. Inf. Syst.* 37 (2) (2019) 1–30.
- [7] Raehyun Kim, Chan Ho So, Minbyul Jeong, Sanghoon Lee, Jinkyu Kim, Jaewoo Kang, Hats: a hierarchical graph attention network for stock movement prediction, *arXiv preprint, arXiv:1908.07999*, 2019.
- [8] Li Wei, Ruihan Bao, Keiko Harimoto, Deli Chen, Jingjing Xu, Qi Su, Modeling the stock relation with graph network for overnight stock movement prediction, in: *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence*, 2021, pp. 4541–4547.
- [9] Daiki Matsunaga, Toyotaro Suzumura, Toshihiro Takahashi, Exploring graph neural networks for stock market predictions with rolling window analysis, *arXiv preprint, arXiv:1909.10660*, 2019.
- [10] Gabriele Corso, Luca Cavalleri, Dominique Beaini, Pietro Liò, Petar Veličković, Principal neighbourhood aggregation for graph nets, *Adv. Neural Inf. Process. Syst.* 33 (2020) 13260–13271.
- [11] Qimai Li, Zhichao Han, Xiao-Ming Wu, Deeper insights into graph convolutional networks for semi-supervised learning, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, 2018.
- [12] Kenta Oono, Taiji Suzuki, Graph neural networks exponentially lose expressive power for node classification, *arXiv preprint, arXiv:1905.10947*, 2019.
- [13] Uri Alon, Eran Yahav, On the bottleneck of graph neural networks and its practical implications, *arXiv preprint, arXiv:2006.05205*, 2020.
- [14] Thomas N. Kipf, Max Welling, Semi-supervised classification with graph convolutional networks, *arXiv preprint, arXiv:1609.02907*, 2016.
- [15] Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, Yaliang Li, Simple and deep graph convolutional networks, in: *International Conference on Machine Learning*, PMLR, 2020, pp. 1725–1735.
- [16] Jiayan Guo, Lun Du, Wendong Bi, Qiang Fu, Xiaojun Ma, Xu Chen, Shi Han, Dongmei Zhang, Yan Zhang, Homophily-oriented heterogeneous graph rewiring, in: *Proceedings of the ACM Web Conference 2023*, 2023, pp. 511–522.
- [17] Xiaohan Li, Jun Wang, Jinghua Tan, Shiyu Ji, Huading Jia, A graph neural network-based stock forecasting method utilizing multi-source heterogeneous data fusion, *Multimed. Tools Appl.* 81 (30) (2022) 43753–43775.
- [18] Deli Chen, Yankai Lin, Wei Li, Peng Li, Jie Zhou, Xu Sun, Measuring and relieving the over-smoothing problem for graph neural networks from the topological view, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, 2020, pp. 3438–3445.
- [19] Wentao Xu, Weiqing Liu, Lewen Wang, Yingce Xia, Jiang Bian, Jian Yin, Tie-Yan Liu, Hist: a graph-based framework for stock trend forecasting via mining concept-oriented shared information, *arXiv preprint, arXiv:2110.13716*, 2021.
- [20] Guowei Song, Tianlong Zhao, Suwei Wang, Hua Wang, Xuemei Li, Stock ranking prediction using a graph aggregation network based on stock price and stock relationship information, *Inf. Sci.* (2023) 119236.
- [21] Vijay Prakash Dwivedi, Xavier Bresson, A generalization of transformer networks to graphs, *arXiv preprint, arXiv:2012.09699*, 2020.
- [22] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, Tie-Yan Liu, Do transformers really perform badly for graph representation?, *Adv. Neural Inf. Process. Syst.* 34 (2021) 28877–28888.
- [23] Qiuyue Zhang, Chao Qin, Yunfeng Zhang, Fangxun Bao, Caiming Zhang, Peide Liu, Transformer-based attention network for stock movement prediction, *Expert Syst. Appl.* 202 (2022) 117239.
- [24] Haoning Xi, Yi Zhang, Yi Zhang, Detection of safety features of drivers based on image processing, in: *18th COTA International Conference of Transportation Professionals*, American Society of Civil Engineers, Reston, VA, 2018, pp. 2098–2109.
- [25] Zhen Wang, Liu Liu, Yiqun Duan, Yajing Kong, Dacheng Tao, Continual learning with lifelong vision transformer, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2022, pp. 171–181.
- [26] Dexiong Chen, Leslie O'Bray, Karsten Borgwardt, Structure-aware transformer for graph representation learning, in: *International Conference on Machine Learning*, PMLR, 2022, pp. 3469–3489.
- [27] Jinsong Chen, Kaiyuan Gao, Gaichao Li, Kun He, Nagphormer: a tokenized graph transformer for node classification in large graphs, in: *The Eleventh International Conference on Learning Representations*, 2022.
- [28] Qitian Wu, Wentao Zhao, Zenan Li, David P. Wipf, Junchi Yan, Nodeformer: a scalable graph structure learning transformer for node classification, *Adv. Neural Inf. Process. Syst.* 35 (2022) 27387–27401.
- [29] Devin Kreuzer, Dominique Beaini, Will Hamilton, Vincent Létourneau, Prudencio Tossou, Rethinking graph transformers with spectral attention, *Adv. Neural Inf. Process. Syst.* 34 (2021) 21618–21629.
- [30] Deyu Bo, Chuan Shi, Lele Wang, Renjie Liao, Specformer: spectral graph neural networks meet transformers, *arXiv preprint, arXiv:2303.01028*, 2023.
- [31] Zhanghao Wu, Paras Jain, Matthew Wright, Azalia Mirhoseini, Joseph E. Gonzalez, Ion Stoica, Representing long-range context for graph neural networks with global attention, *Adv. Neural Inf. Process. Syst.* 34 (2021) 13266–13279.
- [32] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, Jayant Kalagnanam, A time series is worth 64 words: long-term forecasting with transformers, *arXiv preprint, arXiv:2211.14730*, 2023.
- [33] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, Wancai Zhang, Informer: beyond efficient transformer for long sequence time-series forecasting, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, pp. 11106–11115.
- [34] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al., Transformers: state-of-the-art natural language processing, in: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.
- [35] Djork-Arné Clevert, Thomas Unterthiner, Sepp Hochreiter, Fast and accurate deep network learning by exponential linear units (elus), *arXiv preprint, arXiv:1511.07289*, 2015.

- [36] Mahinda Mailagaha Kumbure, Christoph Lohrmann, Pasi Luukka, Jari Porras, Machine learning techniques and data for stock market forecasting: a literature review, *Expert Syst. Appl.* 197 (2022) 116659.
- [37] I.M. Yassin, M.F. Abdul Khalid, S.H. Herman, Ibrahim Pasya, N. Ab Wahab, Zaiki Awang, Multi-layer perceptron (mlp)-based nonlinear auto-regressive with exogenous inputs (narx) stock forecasting model, *Int. J. Adv. Sci. Eng. Inf. Technol.* 7 (3) (2017) 1098–1103.
- [38] Sepp Hochreiter, Jürgen Schmidhuber, Long short-term memory, *Neural Comput.* 9 (8) (1997) 1735–1780.
- [39] Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, Yoshua Bengio, Empirical evaluation of gated recurrent neural networks on sequence modeling, *arXiv preprint, arXiv:1412.3555*, 2014.
- [40] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, et al., Graph attention networks, *Stat* 1050 (20) (2017) 10–48550.
- [41] Fuli Feng, Huimin Chen, Xiangnan He, Ji Ding, Maosong Sun, Tat-Seng Chua, Enhancing stock movement prediction with adversarial training, *arXiv preprint, arXiv:1810.09936*, 2018.
- [42] Qianggang Ding, Sifan Wu, Hao Sun, Jiadong Guo, Jian Guo, Hierarchical multi-scale Gaussian transformer for stock movement prediction, in: *IJCAI*, 2020, pp. 4640–4646.
- [43] Xiao Yang, Weiqing Liu, Dong Zhou, Jiang Bian, Tie-Yan Liu, Qlib: an ai-oriented quantitative investment platform, *arXiv preprint, arXiv:2009.11189*, 2020.
- [44] Clive WJ Granger, Investigating causal relations by econometric models and cross-spectral methods, *Econometrica* (1969) 424–438.
- [45] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al., Pytorch: an imperative style, high-performance deep learning library, *Adv. Neural Inf. Process. Syst.* 32 (2019).